

Clearing Dense Log Piles with Imitation-Bootstrapped Reinforcement Learning

Anonymous Authors

Abstract—Clearing a dense pile of logs by sequential grasping is a contact-rich manipulation problem. The scene contains hundreds of rigid bodies in frictional contact, each grasp reshapes the pile, and effective strategies span long horizons. We study this problem in simulation for a forestry log loader that must clear piles of 200 logs directly from depth-derived point clouds. In our experiments, from-scratch reinforcement learning (RL) with Proximal Policy Optimization (PPO) across several observation formulations does not discover strategies that effectively clear the pile while maintaining grasp quality. Our framework first trains a PointNet-based policy via behavior cloning (BC) on demonstrations from a privileged-state heuristic, then fine-tunes with RL using an asymmetric actor-critic. BC provides the full-trajectory exploration curriculum that RL alone cannot discover, while RL optimizes per-grasp quality beyond what the demonstrator achieves. The resulting policy maintains expert-level clearing (98.2%) while producing 15% more stable grasps than BC and 19% more than the privileged-state heuristic, from depth-derived point clouds alone.

Index Terms—robot learning, multi-object grasping from dense clutter, behavior cloning, reinforcement learning fine-tuning, contact-rich simulation

I. INTRODUCTION

The forestry industry is increasingly looking for autonomous solutions for heavy machinery used in mills and forests to improve safety and productivity [?]. One key subtask carried out by log-loading machines is log-pile clearing: repeatedly grasping multiple logs from a rack or pile and removing them. This setting is challenging because (i) the scene is densely cluttered with hundreds of rigid-body logs in frictional contact, (ii) each grasp disturbs the pile through contact cascades, continuously reshaping the configuration for subsequent attempts, and (iii) no two pile configurations are alike, so a targeting strategy must be robust to diverse arrangements.



Fig. 1: A log loader clearing piles at a millyard infeed deck.

We address the grasp target selection problem: given a raw point cloud derived from a stereo depth camera, where should the crane place and orient the grapple to clear the pile efficiently? We deliberately separate this high-level targeting decision from low-level motion execution. Concretely, we implement a grasp-and-remove simulator in Isaac Lab [?] where each environment step is one complete pick attempt: a policy selects where to grasp, a fixed finite-state machine (FSM) handles execution, and grasped logs are removed rather than deposited, allowing us to measure clearing progress without modeling the log placement dynamics.

Existing work on autonomous log grasping addresses small-scale settings: single-log pick-and-place, small groupings of two to seven logs, or grasp prediction without sequential clearing. We scale to full pile-level clearing with dense frictional contact dynamics among hundreds of logs. To our knowledge, this is the first study of learned sequential grasp targeting at this pile scale. Dense log-pile clearing poses a difficult exploration problem for RL: reward signal is sparse across long episodes of sequential grasps, and the high-dimensional point-cloud observation space compounds the challenge. Our approach is to first train a policy via behavior cloning (BC) on demonstrations from a privileged-state heuristic, then fine-tune with RL. The demonstration data spans the complete clearing trajectory, giving the RL agent the distributional coverage it cannot acquire through its own exploration.

Our main contributions are:

- A BC→RL fine-tuning pipeline for grasp target selection in dense log piles from raw point clouds. BC provides trajectory-level coverage of pile states, while RL optimizes per-grasp quality beyond what the demonstrator achieves.
- A grasp-and-remove task formulation for dense log-pile clearing at scale (200 logs, 30-grasp horizons), with an FSM-based controller that isolates learned targeting from execution.
- A four-method comparison (heuristic, RL, BC, BC→RL) showing that from-scratch RL underperforms regardless of observation type, BC achieves near-expert clearing from raw point clouds, and BC→RL improves grasp stability by 15% over BC and 19% over the heuristic while maintaining clearing performance.

II. RELATED WORK

A. Grasping in Clutter

Learning-based grasp detection has progressed from single isolated objects to cluttered, multi-object scenes. Deep

grasp prediction networks predict grasp pose directly from depth images [?], [?], and large-scale synthetic datasets have enabled robust grasp planning without real-world training data [?]. Sequential decluttering has been studied in bin picking, where learned policies account for how each pick changes the heap [?] and pushing is combined with grasping to handle dense clutter [?]. Applications extend to natural-resource handling such as forestry log loading [?].

However, these works typically involve heterogeneous objects of varied shape and size in loose arrangements, where clutter arises from random placement rather than dense packing. The grasping primitive is almost always a precision or pinch grasp that isolates a single target object. Our setting differs in important ways: we perform power grasps from dense piles of cylindrical logs of uniform geometry that are initially stacked in ordered layers. The homogeneity and tight packing mean that logs are in extensive frictional contact with their neighbors, and each grasp attempt (which captures multiple logs simultaneously) disturbs the pile, causing settling, rolling, and potential knock-offs that reshape the configuration for all subsequent attempts. For 3D perception of unordered point sets, PointNet [?] and its variants provide a natural encoder architecture.

B. Learning for Log Grasping and Forestry Crane Control

A growing body of work applies learning methods to log grasping and forestry crane automation, though existing approaches address smaller-scale settings than the one considered here.

For grasp prediction, Wallin et al. [?] use RL with virtual visual servoing for multi-log grasping from small groupings of 2–5 logs in simulation, separating image segmentation from Cartesian crane control; their agent achieves 95% success but the pile dynamics are limited to a handful of objects. Fälldin et al. [?] train a U-Net CNN on synthetic RGB-D images to predict grasp poses for small clusters of up to 7 logs, with an open-loop kinematic controller; their work focuses on grasp prediction from synthetic data and does not address sequential clearing or pile-scale dynamics. Ayoub et al. [?] propose a CNN-based grasp planner for a log-loading forestry machine that predicts a grasp configuration from a depth image for small piles, validated both in simulation and on a commercial log loader retrofitted for autonomy.

For crane control, Vu et al. [?] present a MuJoCo-based forestry crane simulator and benchmark RL velocity controllers for grasping single logs of varying diameter (simulation only). Kurinov et al. [?] train an RL agent for the full single-log pick-and-place pipeline (locate, grasp, transport, load) in Isaac Gym simulation, achieving 94% success across parallel environments but with one log at a time. Jebellat et al. [?] use dynamic programming to design anti-sway trajectories for a log loader end effector. These works address individual subtasks but none tackle sequential clearing of a dense pile at the scale considered here.

More broadly, using demonstrations to initialize RL policies has been shown to overcome exploration challenges

in contact-rich manipulation [?], [?], and asymmetric actor-critic architectures (where the critic receives privileged state information unavailable to the actor) have proven effective for learning from high-dimensional visual observations [?]. To our knowledge, no prior work combines these ideas for sequential pile clearing from point clouds.

III. TASK FORMULATION

A. Grasp-and-Remove Formulation

Each environment step corresponds to a complete grasp attempt: a policy selects a grasp target, the FSM executes the approach, close, and lift, and grasped logs are removed from the scene. This formulation is shared by all methods (heuristic, BC, RL, and BC+RL). The state $s_t \in \mathcal{S}$ is the current pile configuration (observed via point cloud or privileged poses; Section III-B). A grasp target is defined by the tuple

$$\mathbf{g} = (x, y, z, \psi), \quad (1)$$

where (x, y, z) is the grapple position in the crane base frame and ψ is the grapple yaw angle about the vertical axis. Because a cylindrical log at yaw ψ is identical to the same log at $\psi + \pi$, the policy represents ψ via the π -periodic encoding $(\cos 2\psi, \sin 2\psi)$ [?], giving a 5D action $a_t \in \mathcal{A} \subset \mathbb{R}^5$ (Section III-C). The transition $T(s_{t+1}|s_t, a_t)$ is determined by the physics simulation and FSM execution. After each grasp attempt, an outcome evaluation measures the number of logs grasped, grapple–log alignment, and grapple stability (Section IV-B). For RL training, this outcome is used as a scalar reward $r_t = R(s_t, a_t)$, and the full structure is treated as an episodic Markov decision process $(\mathcal{S}, \mathcal{A}, T, R, \gamma)$ with discount factor γ , where a policy π maximizes the expected discounted return over a horizon of H grasp attempts:

$$\max_{\pi} \mathbb{E}_{\pi} \left[\sum_{t=0}^{H-1} \gamma^t r_t \right]. \quad (2)$$

An episode terminates when no logs remain on the rack or after a fixed horizon of $H = 30$ grasp attempts. A log leaves the rack either by being grasped and removed or by being knocked outside the workspace bounds (Section IV-B); both reduce the remaining count for termination purposes, but only intentionally grasped logs are credited toward the clearing rate metric (Section VI-B). The horizon is chosen to exceed the minimum needed: the grapple can hold up to 20 logs per grasp, and the heuristic baseline clears 200-log piles in approximately 20 attempts, so $H = 30$ provides a 50% margin while keeping episodes tractable.

B. Observations

The policy observes a raw point cloud (PC) of the scene. A depth camera mounted on the crane renders the scene from above the rack, producing a depth image D_t . The simulated camera uses a pinhole model with parameters matching the Stereolabs ZED X in wide-angle configuration¹.

¹<https://www.stereolabs.com/products/zed-x>

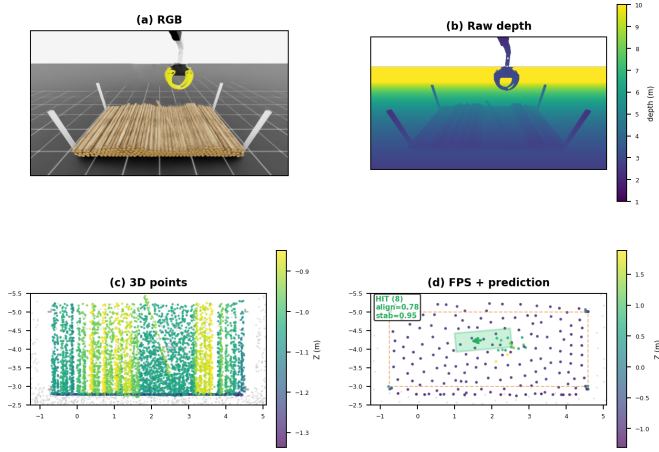


Fig. 2: Observation pipeline for a single grasp: RGB, raw depth, 3D back-projection in the crane base frame, and FPS-downsampled point cloud with the predicted grasp target and action-space bounds (dashed box).

The observation is constructed in three steps (Fig. 6). First, each pixel (u, v) with a valid depth reading within the sensor range $[d_{\min}, d_{\max}]$ is back-projected to a 3D point in the camera frame using the intrinsic matrix $\mathbf{K} \in \mathbb{R}^{3 \times 3}$:

$$\mathcal{P}_c = \{ \mathbf{K}^{-1}(u, v, 1)^\top \cdot D_t(u, v) \mid d_{\min} \leq D_t(u, v) \leq d_{\max} \}. \quad (3)$$

These camera-frame points are then transformed to the world frame $\mathcal{P}_w$ using the known camera extrinsics. Second, each point $\mathbf{p} \in \mathcal{P}_w$ is transformed into the crane base frame using the base-frame rotation $\mathbf{R}_b$ and translation $\mathbf{t}_b$:

$$\mathcal{P}_b = \{ \mathbf{R}_b^\top (\mathbf{p} - \mathbf{t}_b) \mid \mathbf{p} \in \mathcal{P}_w \}. \quad (4)$$

Third, Farthest Point Sampling (FPS) downsamples $\mathcal{P}_b$ to a fixed-size set of $N_p = 1024$ points, yielding the observation:

$$\mathbf{o}_t = \text{FPS}(\mathcal{P}_b, N_p) \in \mathbb{R}^{N_p \times 3}. \quad (5)$$

If the cloud contains fewer than N_p points (e.g., near the end of an episode), the remaining rows are zero-padded. Crucially, no semantic segmentation is applied: the raw point cloud includes all surfaces visible to the depth camera (logs, rack, ground). The PointNet encoder must implicitly learn to attend to task-relevant geometry. We compare raw and segmented point clouds in Section VII.

As an ablation, we also evaluate policies that observe privileged per-object state: the top- K log poses sorted by height (highest first), each represented as (x, y, z, ψ) in the crane base frame. Removed logs are excluded and remaining slots are padded with -1 . With $K=32$ this yields a compact $4K = 128$ dimensional vector. This baseline isolates the effect of the perception gap: any performance difference between pose and point cloud policies is attributable to the observation representation rather than the learning algorithm.

C. Actions and Workspace Scaling

The policy outputs a 5D vector $\tilde{\mathbf{a}}_t \in \mathbb{R}^5$ (before squashing). The first three components encode position and the remaining two encode grapple yaw via a cosine-sine pair.

Each position component is squashed via the hyperbolic tangent $\tanh$ and then linearly mapped to the per-environment workspace bounds to produce the target coordinate p_i :

$$p_i = \frac{(\tanh \tilde{a}_i + 1)}{2} (b_i^{\max} - b_i^{\min}) + b_i^{\min}, \quad i \in \{x, y, z\}, \quad (6)$$

where $b_i^{\min}, b_i^{\max}$ are the rack-aligned workspace limits for each axis. These bounds are derived from the rack geometry with margins along each axis. The resulting workspace is approximately $2.0 \times 5.2 \times 1.0$ m in the crane base frame. These bounds define the reachable target space; logs knocked outside them are automatically removed and counted separately (Section IV-B). The heuristic expert (Section V-A) operates within the same bounds.

The remaining two components encode grapple yaw via the π -periodic pair $(\cos 2\psi, \sin 2\psi)$ [?]. The yaw is recovered as $\psi = \frac{1}{2} \text{atan2}(\tanh \tilde{a}_5, \tanh \tilde{a}_4) \in [-\frac{\pi}{2}, \frac{\pi}{2}]$. For BC training, expert targets use the same encoding (positions via arctanh , yaw as $(\cos 2\psi, \sin 2\psi)$).

IV. ENVIRONMENT AND CONTROLLER

A. Low-level Execution FSM

All methods share the same fixed FSM controller, which runs at the control rate (60 Hz, i.e., 120 Hz physics with $2 \times$ decimation) inside each environment step. Given a grasp target (x, y, z, ψ) , produced by the heuristic (Section V-A) or a learned policy (BC or RL), the FSM executes a complete grasp cycle.

The crane arm has four actuated joints (base-mast, boom, stick, and telescope) followed by a two-link passive chain that suspends the grapple from the telescope tip. The two passive joints are unactuated revolute joints with low stiffness and damping, modeling the gravity-driven hanging behavior of a real forestry crane's grapple attachment. Below the passive chain, a yaw joint actively rotates the grapple about the vertical axis, and two gripper joints control the jaw opening.

In this kinematic structure, the IK controller commands the top of the passive chain, but the grapple body hangs ~ 0.4 m below it and swings like a pendulum during arm movements. Consequently, all grasp targets are specified for the grapple body and then offset vertically to the IK target link. The FSM uses a two-stage convergence check at each phase: it first waits for the IK target link to reach its target within $\epsilon_{\text{up}} = 0.10$ m, then waits for the grapple body to settle within $\epsilon_{\text{bg}} = 0.25$ m. Both must hold for $N_{\text{dwell}} = 30$ consecutive timesteps before the phase advances, ensuring that pendulum oscillations have damped before proceeding.

The FSM implements a five-phase grasp cycle: HOVER (move above target with $z_{\text{clear}} = 2.5$ m clearance), ALIGN (rotate grapple to target yaw ψ), DESCEND (lower at 30% speed to minimize disturbance), CLOSE (close jaws via discrete stepping), and LIFT (raise at 50% speed, evaluate outcome).

per Section IV-B, remove grasped logs, open jaws, repeat). A per-phase timeout (100 timesteps) forces advancement if convergence is not reached.

The arm is controlled using Isaac Lab’s task-space Differential IK controller [?] (damped least-squares, position command type), which computes joint position targets from the current end-effector pose and Jacobian. The grapple yaw joint is controlled independently via a PD position controller. The grapple jaws are controlled using a PD velocity law with saturation and discrete stepping to avoid abrupt contact forces.

B. Outcome Evaluation

After lifting (LIFT), the FSM holds the grapple at hover height and evaluates grasp outcome by measuring proximity and orientation of logs relative to the grapple:

Logs grasped (g): the number of active logs (i.e., logs not yet removed from the scene) whose centers lie within a proximity radius of the grapple (default 1.5 m).

Alignment (α): for each grasped log i , we compute per-log alignment α_i from the angle between the grapple’s jaw axis $\hat{y}_g$ and the log’s length axis $\hat{y}_{\ell_i}$ via their dot product ($\hat{y}_g^\top \hat{y}_{\ell_i} = \cos \theta_\alpha$, where θ_α is the misalignment angle). Because a cylindrical log has no preferred length direction, the even exponent absorbs the sign; raising to the 8th power creates a sharp dropoff that penalizes moderate misalignment:

$$\alpha_i = (\hat{y}_g^\top \hat{y}_{\ell_i})^8, \quad \alpha = \frac{1}{g} \sum_{i=1}^g \alpha_i \in [0, 1]. \quad (7)$$

Stability (ς): the grapple’s levelness after lifting, measured as $\hat{z}_g^\top \hat{z}_w = \cos \theta_\varsigma$, where θ_ς is the tilt angle between the grapple’s up vector and the world vertical. Clamping to zero excludes the degenerate case of an inverted grapple; raising to the 4th power ensures high scores only when the grapple is nearly level:

$$\varsigma = (\max(0, \hat{z}_g^\top \hat{z}_w))^4 \in [0, 1]. \quad (8)$$

Logs identified as grasped are marked inactive and teleported out of the scene. Logs that leave the workspace bounds during a grasp are removed and excluded from the grasp count g . Figure 2 visualizes the shaped score curves for alignment and stability.

We implement the environment in Isaac Lab and run PhysX GPU dynamics for scalable data collection. Log and contact parameters (Table I) are based on measurements of representative forestry logs and published friction coefficients for oak on oak [?]. We use conservative contact modeling (simplified collision geometry on the grapple, tuned contact and rest offsets, and damping limits for logs) to reduce interpenetration and numerical jitter while maintaining high throughput.

V. BASELINE AND LEARNED POLICIES

A. Heuristic Expert

We implement a hand-tuned heuristic expert that serves both as a competitive baseline and as the demonstration source for BC. The heuristic has access to privileged information (the

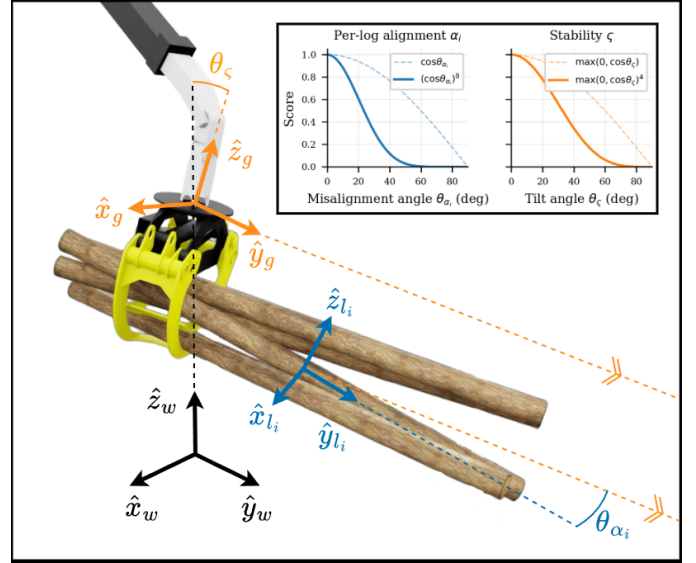


Fig. 3: Coordinate frames used to compute alignment (α) and stability (ς).

TABLE I: Simulation and controller parameters. Actuator gains are implicit PhysX joint-drive values; IK gains are the PD velocity controller used by the FSM.

Crane link masses		Simulation	
Mast / Boom / Stick	290 / 172 / 70 kg	Physics timestep (Δt)	1 / 120 s
Telescope / Passive chain	25 / 9 kg	Control decimation	$2 \times (60 \text{ Hz})$
Joint limits		Actuator gains (k_p / k_d)	
Base-mast (revolute)	$\pm 100^\circ$	Main arm joints	10^6 / 10^5
Boom (revolute)	-22° to $+75^\circ$	Passive chain	300 / 500
Stick (revolute)	-177° to $+2^\circ$	Grapple yaw	5×10^4 / 2×10^4
Telescope (prismatic)	0.13–1.80 m	Grapple tongs	0 / 500
Grapple (revolute)	$\pm 180^\circ$	IK PD gains (k_p / k_d)	
Log properties		Arm joints	4.0 / 1.2
Diameter / length	0.11 / 2.45 m	Grapple tongs	12.0 / 1.0
Mass	15 kg	FSM parameters	
μ_s / μ_k	0.62 / 0.48	Hover clearance	2.5 m
Angular damping	0.05	Pos. tol. (up. pas. / grap.)	0.10 / 0.25 m
Contact / rest offset	0.02 / 0.0 m	Dwell count	30 steps
Max depenetration vel.	3.0 m/s	Phase timeout	100 steps

position and orientation of every active log from the physics engine) and uses a deterministic top-of-pile targeting strategy:

At each pick cycle, the expert selects the topmost active log (highest z -coordinate), transforms its center into the crane base frame as the (x, y, z) target, and aligns the grapple yaw ψ with the log’s length axis (choosing the orientation requiring minimal joint rotation). Yaw alignment is recomputed in the ALIGN phase for closed-loop tracking. This strategy is effective because the highest log is typically the most accessible, but does not adapt in the endgame when few scattered logs remain. Despite this, the expert achieves strong performance across all metrics (Table II).

B. Behavior Cloning

We use BC [?] to train a policy from expert demonstrations. We collect demonstration pairs (o_t, a_t^*) from successful expert

grasps only (failed grasps are discarded), where o_t is the point cloud observation and a_t^* is the expert action in the pre-squashing space (positions via arctanh , yaw via $\cos 2\psi, \sin 2\psi$ as described in Section III-C).

The policy uses a PointNet encoder [?]. A shared per-point MLP $h : \mathbb{R}^3 \rightarrow \mathbb{R}^{256}$ (layers $3 \rightarrow 64 \rightarrow 128 \rightarrow 256$ with batch normalization and ELU activations) maps each point independently; a symmetric max-pool aggregates the per-point features, and a post-pooling fully connected layer f_{fc} ($256 \rightarrow 256$, BN, ELU) yields the global latent vector:

$$\mathbf{z} = f_{fc}\left(\max_{i=1,\dots,N_p} h(o_t^{(i)})\right) \in \mathbb{R}^{256}, \quad (9)$$

where $o_t^{(i)} \in \mathbb{R}^3$ is the i -th point in the observation. An actor MLP $\mu_\theta : \mathbb{R}^{256} \rightarrow \mathbb{R}^5$ (layers $256 \rightarrow 128 \rightarrow 64 \rightarrow 5$) maps the latent to the action $\tilde{a}_t = \mu_\theta(\mathbf{z})$. For the privileged-pose ablation, we replace the PointNet encoder with a three-layer MLP ($128 \rightarrow 256 \rightarrow 128 \rightarrow 64$) with ELU activations.

We apply dropout (rate 0.2) after the encoder. The BC objective minimizes the mean-squared error between the predicted and expert actions over a dataset $\mathcal{D}$ of successful grasps:

$$\mathcal{L}_{BC} = \frac{1}{|\mathcal{D}|} \sum_{(o, a^*) \in \mathcal{D}} \|\mu_\theta(\mathbf{z}) - a^*\|^2, \quad (10)$$

where $\mathbf{z}$ is the encoder output for observation o . We optimize with AdamW [?] (learning rate 10^{-4} , weight decay 10^{-4}) and ReduceLROnPlateau scheduling (factor 0.5, patience 10 epochs). The training dataset contains $\sim 20,700$ successful-grasp samples ($\sim 85/15$ train/validation split). The raw PC model is selected by minimum validation loss (0.101 MSE, epoch 190) with early stopping (patience 30 epochs).

C. Reinforcement Learning

We train policies with PPO [?] using the RSL-RL library [?]. PPO optimizes a stochastic Gaussian policy with parameters θ :

$$\pi_\theta(a | s) = \mathcal{N}(\mu_\theta(s), \text{diag}(\sigma^2)), \quad (11)$$

where μ_θ outputs the mean action and $\sigma \in \mathbb{R}^{|A|}$ is a learnable standard deviation vector initialized to $\sigma_0 \mathbf{1}$. A separate critic $V_\phi(s)$ estimates the state-value function; advantages are computed via Generalized Advantage Estimation [?].

The actor uses the same PointNet encoder and actor MLP as the BC policy; the critic uses an independent PointNet encoder with its own MLP head. For the privileged-pose ablation, both actor and critic are separate MLPs with hidden dimensions $[256, 128, 64]$ and ELU activations. Key hyperparameters: $\sigma_0 = 1.0$, clipping ratio $\epsilon = 0.2$, entropy coefficient 0.01, learning rate 3×10^{-4} with adaptive KL-based scheduling (target KL 0.01), GAE with $\lambda = 0.95$, discount factor $\gamma = 0.99$, 4 steps per environment per iteration, 5 learning epochs with 4 mini-batches per epoch, and gradient clipping at norm 1.0. Each iteration collects $N_{\text{env}} \times 4$ grasp cycles (one step per cycle) across all parallel environments and performs one PPO update; total grasp experience after k iterations is

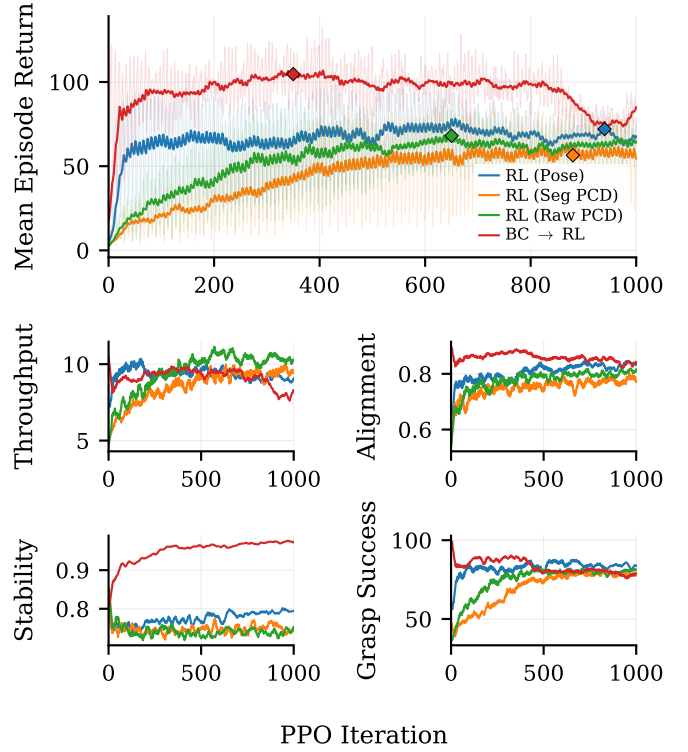


Fig. 4: PPO training curves. Mean episode return vs. PPO iteration (bold: 20-iteration rolling average; shading: raw). Diamond markers indicate the evaluation checkpoint. All three from-scratch RL variants plateau below full clearing (Table II). BC $\rightarrow$ RL reaches the highest return but degrades after ~ 500 iterations, motivating early stopping. Bottom panels decompose the reward into per-grasp components.

therefore $4kN_{\text{env}}$. The outcome components (g, α, ς) from Section IV-B define the scalar reward:

$$r = g \cdot \alpha \cdot \varsigma, \quad (12)$$

for successful grasps ($g > 0$), and $r = -1$ for failed grasps. The reward is thus the product of throughput, alignment, and stability; a high reward requires many logs grasped with good alignment and a level grapple. The reward is entirely per-grasp. Global pile-clearing ability must emerge from the policy's targeting strategy rather than from explicit reward incentives.

D. BC-Initialized RL Fine-Tuning

To combine BC's full-trajectory coverage with RL's ability to optimize per-grasp metrics, we initialize a PPO agent from the trained BC policy and fine-tune with on-policy experience [?], [?].

The BC actor weights (PointNet encoder and actor MLP) are loaded into the PPO actor (Fig. 4); the critic is initialized randomly and the optimizer starts fresh. We use an asymmetric actor-critic [?]: the actor observes the raw point cloud (same as BC), while the critic is a trainable MLP that receives the privileged 128D pose state vector (top- K log poses).

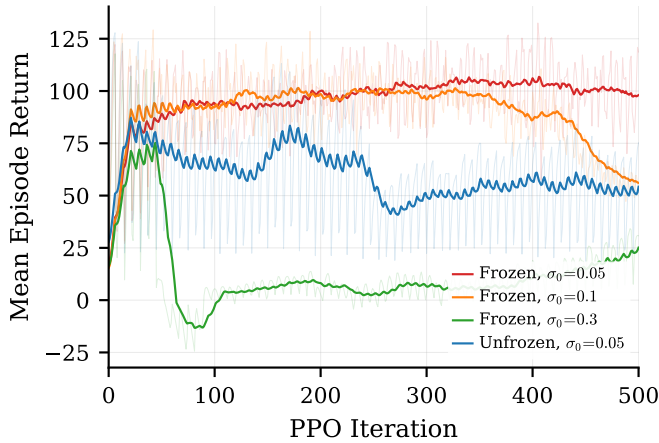


Fig. 5: BC+RL training sensitivity. Mean episode return vs. PPO iteration for four configurations varying encoder freezing and initial policy noise σ_0 .

This asymmetry allows the critic to learn a more accurate value function from compact state information, while the actor retains the vision-based input it will use at deployment.

We freeze the BC-trained PointNet encoder during RL fine-tuning. The asymmetric critic cannot propagate gradients through the encoder, so an unfrozen encoder drifts without constraint; fine-tuning the full network causes training collapse (Fig. 7).

All other PPO hyperparameters remain identical to the from-scratch RL configuration (Section V-C); the only changes are the frozen encoder and a reduced initial policy noise $\sigma_0 = 0.05$ (vs. 1.0 for from-scratch RL), which limits exploration to small perturbations around the BC mean.

Higher noise causes gradual degradation ($\sigma_0 = 0.1$) or rapid collapse ($\sigma_0 = 0.3$), as shown in Fig. 7.

VI. EXPERIMENTS

A. Setup

All experiments use the same crane model, rack geometry, and FSM controller, and run on a single NVIDIA RTX 4070 Laptop GPU (8 GB VRAM). Training uses 32 parallel environments for pose-based policies and 20 for point cloud policies (Fig. 5). Piles contain 200 cylindrical logs spawned in layered grid patterns. The log count is fixed across all experiments so that clearing metrics are directly comparable between methods; pile variation (Table II) randomizes log count (20–200), grid dimensions, spacing (± 5 –15%), layer offsets, and per-log placement jitter to test generalization.

Each episode runs for $H = 30$ grasp attempts (Section III-A). We evaluate over 100 episodes on identical random seeds across 20 parallel environments; all metrics are per-episode mean $\pm$ standard deviation. Training reward curves (Fig. 3) show single training runs; the reported variability comes from averaging over episodes within a run rather than across independent seeds.

B. Metrics

We report metrics derived from the outcome tuple (g, α, ς) (Section IV-B), all as per-episode mean $\pm$ std: *throughput* (mean logs per successful grasp), *alignment* (grasp-count-weighted mean α), *stability* (mean ς over successful grasps), *grasp success rate* (fraction with $g > 0$), *clearing rate* (fraction of logs intentionally removed), and *knocked-off percentage* (fraction knocked outside bounds).

VII. RESULTS

Table II summarizes the evaluation metrics. All four methods share the same controller, pile configuration, and evaluation protocol; the only differences are the targeting policy and its observation. From-scratch RL uses a symmetric actor-critic in which both actor and critic process observations through independent encoders (PointNet for PC variants, MLPs for the pose ablation). BC+RL uses an asymmetric actor-critic [?] in which the actor retains the frozen BC-trained PointNet encoder while the critic is a trainable MLP that receives the privileged state vector (Section V-D).

A. BC+RL Performance

BC+RL fine-tuning improves grasp stability from $\varsigma = 0.824$ (BC) to 0.947, also surpassing the privileged-state heuristic (0.797), while preserving full-trajectory clearing (Table II). Alignment remains comparable and throughput is slightly lower (8.65 vs. 9.32).

Both the frozen encoder and conservative noise are critical. Figure 7 compares four BC+RL training configurations: freezing the encoder with $\sigma_0 \in \{0.05, 0.1, 0.3\}$ and fine-tuning the full network with $\sigma_0 = 0.05$. Only the frozen-encoder, $\sigma_0 = 0.05$ configuration sustains clearing performance through training; higher noise causes gradual ($\sigma_0 = 0.1$) or rapid ($\sigma_0 = 0.3$) collapse, and unfreezing the encoder causes immediate feature drift regardless of noise level.

B. BC Performance

BC achieves 98.5% clearing using only raw point clouds, comparable to the privileged-state heuristic (99.0%, Table II). Conditional per-grasp metrics closely match the heuristic, with the largest gap in grasp success rate (84.5% vs. 88.7%).

C. RL Performance

From-scratch RL achieves competitive per-grasp reward during training (Fig. 3): RL (Pose) converges to a mean episode reward of ~ 95 – 97 within 80 iterations, while RL (PC) reaches ~ 85 – 90 after 800+ iterations. However, high training reward does not translate to pile-clearing performance.

Three RL variants (privileged pose, segmented PC, raw PC) all achieve roughly similar clearing (Table II). In contrast, observation type does matter for BC: raw PC outperforms segmented PC in both clearing and success rate (Table II). The best RL variant (Raw PC) achieves only 53% clearing with 53.3% grasp success, compared to 98.5% and 84.5% for BC (Table II).

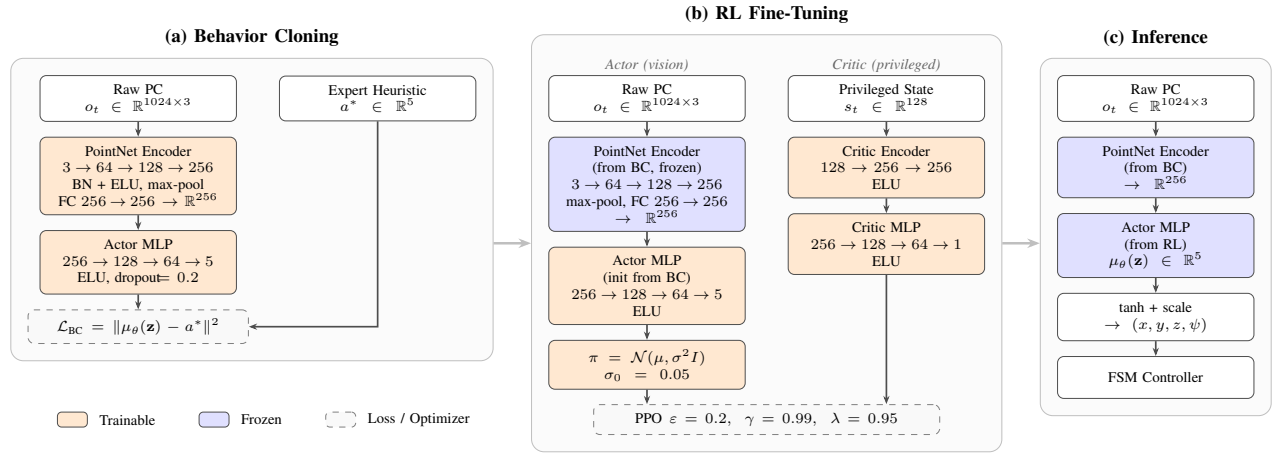


Fig. 6: The BC→RL pipeline. (a) The PointNet encoder and actor MLP are trained end-to-end on expert demonstrations with MSE loss. (b) The encoder is frozen and the actor MLP is fine-tuned with PPO alongside a state-based asymmetric critic. (c) At inference the critic is discarded; the encoder and actor run deterministically.

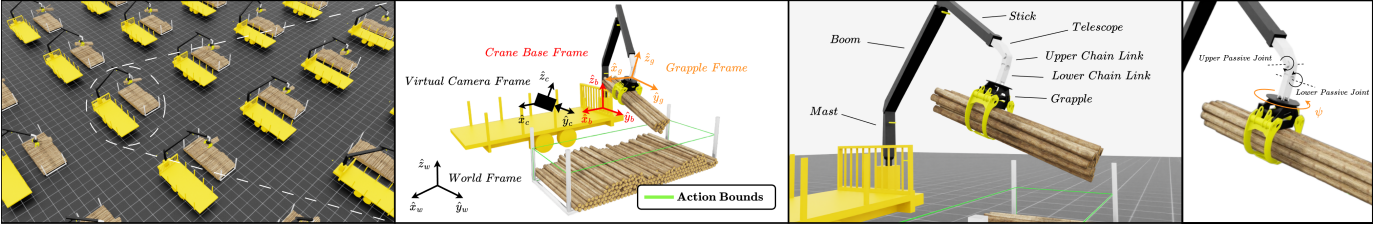


Fig. 7: The grasp-and-remove crane environment. From left to right: parallelized training environments in Isaac Sim, each containing a log loader crane above a rack of 200 logs in randomized configurations; a single environment showing the relevant coordinate frames and the action-space bounding box; the crane link structure; the passive joints and grapple yaw.

VIII. DISCUSSION

From-scratch RL plateaus near 53% clearing regardless of observation type, while BC and BC+RL maintain clearing rates comparable to the heuristic (Table II). The near-identical clearing of BC and BC+RL confirms that RL fine-tuning preserves the expert’s clearing ability while improving grasp stability. Under the combined reward ($g \cdot \alpha \cdot \varsigma$), the RL policy achieves high per-grasp scores without clearing the full pile, converging to a strategy that fails as the pile thins. This is not a perception limitation (privileged pose fares no better) but an exploration one: the policy never encounters the pile-state diversity that arises during complete clearing. BC initialization provides the trajectory-level coverage that from-scratch RL lacks, spanning the full sequence from dense stacks to sparse endgames [?]. RL fine-tuning improves per-grasp execution without rediscovering the global targeting strategy. The encoder must remain frozen (the asymmetric critic provides no gradient to the visual pathway, Section V-D) and low noise ($\sigma_0 = 0.05$) prevents overwriting the learned policy.

The success of raw point clouds has practical implications for sim-to-real transfer: operating directly on depth-derived point clouds simplifies the perception pipeline and reduces potential failure modes. All three methods maintain near-

complete clearing under pile variation (Table II); throughput and grasp success decrease on varied piles, as irregular configurations present fewer easy multi-log grasps, and the learned policies drop more than the heuristic, likely because demonstrations were collected on fixed 200-log piles. Our design has limitations: grasp success uses proximity rather than contact constraints, removed logs are teleported (omitting deposition), and all experiments are in simulation. Architectures exploiting local geometry (PointNet++ [?], point transformers) may improve endgame targeting. Despite these simplifications, the environment captures the essential challenges of sequential multi-log grasping.

IX. CONCLUSION

We presented a grasp-and-remove formulation for dense log-pile clearing at scale and compared four targeting methods on a shared controller. From-scratch RL clears only half the pile regardless of observation type; BC from raw point clouds matches the privileged-state heuristic; BC→RL fine-tuning improves grasp stability by 15% over BC while maintaining near-complete clearing. Future work will explore sim-to-real transfer and richer point cloud architectures.

ACKNOWLEDGMENT

[Redacted for anonymity]

TABLE II: Evaluation metrics (mean $\pm$ std, 100 episodes). Upper rows: fixed 200-log piles. Lower rows: randomized pile variation (log count 20–200, grid dimensions, spacing, offsets, and placement jitter). The heuristic uses privileged pose state. Best per column among point-cloud methods in **bold**.

Method	Obs.	g	α	ς	Grasp succ. (%)	Clear rate (%)	Knocked off (%)
Heuristic	Pose	9.94 ± 1.17	0.918 ± 0.031	0.797 ± 0.018	88.7 ± 7.8	99.0 ± 1.2	1.0 ± 1.1
RL	Pose (128D)	8.84 ± 1.68	0.908 ± 0.040	0.970 ± 0.021	38.5 ± 13.5	49.4 ± 14.8	0.0 ± 0.2
RL	Seg. PC	7.04 ± 1.06	0.726 ± 0.077	0.776 ± 0.042	53.7 ± 10.9	55.2 ± 5.4	0.9 ± 0.7
RL	Raw PC	6.78 ± 1.01	0.758 ± 0.072	0.815 ± 0.039	53.3 ± 8.7	53.0 ± 3.9	0.6 ± 0.6
BC	Seg. PC	9.64 ± 1.05	0.907 ± 0.028	0.816 ± 0.020	70.3 ± 10.6	96.5 ± 4.7	0.9 ± 1.0
BC	Raw PC	9.32 ± 1.12	0.904 ± 0.037	0.824 ± 0.025	84.5 ± 9.8	98.5 ± 5.1	0.7 ± 0.7
BC→RL	Raw PC	8.65 ± 0.97	0.906 ± 0.027	0.947 ± 0.016	86.3 ± 8.5	98.2 ± 1.5	1.2 ± 0.8
<i>Pile variation</i>							
Heuristic	Pose	8.28 ± 1.96	0.889 ± 0.074	0.789 ± 0.024	86.3 ± 11.6	98.2 ± 4.9	1.3 ± 1.6
BC	Raw PC	7.55 ± 2.02	0.860 ± 0.093	0.820 ± 0.029	72.8 ± 17.7	98.4 ± 2.5	1.2 ± 1.5
BC→RL	Raw PC	6.97 ± 1.63	0.861 ± 0.081	0.922 ± 0.037	76.0 ± 14.9	98.5 ± 1.5	1.3 ± 1.4